FICHA DE ACOMPANHAMENTO – APS 2026

Universidade Paulista – UNIP | Ciência da Computação – 7º e 8º semestres
Disciplina: Desenvolvimento de Sistemas Distribuídos
Etapa 4 – Gerador de carga em Python e linha de base

1. IDENTIFICAÇÃO

Grupo nº / Turma		**Data**	
Domínio da aplicação			
Repositório de código			
Ambiente de medição	Máquina do cluster: CPU _____ núcleos ____ RAM ____ GB | Cluster: _____ nós ____ | Gerador na mesma máquina? ( ) Sim ( ) Não		
Rota(s) medida(s)			

2. INTEGRANTES DO GRUPO

#	NOME COMPLETO	RA	RESPONSABILIDADE NESTA ETAPA
1			
2			
3			
4			
5			

3. OBJETIVO DA ETAPA

Até a Etapa 3 o grupo **afirmou** que o sistema distribui carga e escala; a partir desta etapa ele passa a **medir**. O grupo deve escrever um **gerador de carga autoral em Python**: um programa que simula muitos clientes (sensores e usuários) enviando requisições ao mesmo tempo para o Ingress, cronometra cada resposta e anota qual pod a atendeu. Com ele, o grupo executa o **experimento de linha de base**: com **uma única réplica** de cada serviço, aumenta a carga em degraus até o sistema **saturar**, ou seja, até a latência p95 disparar e surgirem erros, e identifica qual recurso esgotou primeiro. O número obtido ("uma réplica aguenta X req/s com p95 de Y ms") é a **régua** com que todos os experimentos da Etapa 5 serão comparados. **Detalhamento no Anexo A.**
(Documento da APS 2026, itens II.3 – Gerador de carga em Python – e II.4 – Linha de base; cronograma II.6, item 5.)

4. CHECKLIST DOS REQUISITOS OBRIGATÓRIOS

Preenchimento pelo grupo; conferência pelo professor durante a demonstração.

#	REQUISITO	EVIDÊNCIA / COMANDO	SIM	NÃO
1	Script **autoral** em Python no repositório (Locust/k6 só como comparação)	arquivo no repositório	( )	( )
2	Concorrência e duração configuráveis por linha de comando	`-c` e `-d` (ou equivalentes)	( )	( )
3	Modo de rampa (carga crescente ao longo do tempo)	`--rampa` / `--degraus`	( )	( )
4	Latência de **cada** requisição registrada; cálculo de p50, p95 e p99	resumo do gerador	( )	( )
5	Vazão (respostas por segundo) e taxa de erro	resumo do gerador	( )	( )
6	Tempo limite por requisição; *timeout* e falha de conexão contados como erro	`--timeout`	( )	( )
7	Exportação em CSV ou JSON (por requisição e resumo)	arquivos em `resultados/`	( )	( )
8	Registro do pod que atendeu cada requisição (cabeçalho `X-Pod` ou log)	contagem por pod	( )	( )
9	Série temporal segundo a segundo, com a hora do relógio (para a Etapa 5)	arquivo de série	( )	( )
10	Linha de base com **1 réplica** de cada serviço e o HPA removido	`kubectl get deploy,hpa`	( )	( )

11	Degraus de carga até a latência p95 degradar e/ou surgirem erros	tabela da seção 5	( )	( )
12	Recurso que satura identificado com dados	`kubectl top pod` durante a carga	( )	( )
13	Gráfico de latência e vazão por nível de carga	PNG no repositório	( )	( )

5. LINHA DE BASE (PREENCHIMENTO PELO GRUPO)

Uma réplica de cada serviço. Rota: _____ Duração de cada degrau: ____ s Aquecimento descartado: ____ s

TRABALHADORES	REQUISIÇÕES	VAZÃO (REQ/S)	P50 (MS)	P95 (MS)	P99 (MS)	ERROS (%)	CPU DO POD (`TOP`)
1							
2							
4							
8							
16							
32							
64							

Ponto de saturação	a partir de _____ trabalhadores | vazão máxima _____ req/s | p95 nesse ponto _____ ms
O que saturou primeiro	( ) CPU do serviço (limit) ( ) banco ( ) fila/cache ( ) máquina hospedeira ( ) o próprio gerador ( ) outro: _____

6. VERIFICAÇÃO AO VIVO

Solicitada pelo professor durante a aula. Anotar o resultado observado.

AÇÃO	COMANDO	RESULTADO OBSERVADO
Rodar o gerador por 20 s pelo Ingress, com 8 trabalhadores	`python loadgen.py URL -c 8 -d 20`	_____ req/s p95 _____ ms
Mostrar a distribuição por pod com 2 réplicas	resumo “por pod”	
Abrir o CSV por requisição e localizar a maior latência	`-requisicoes.csv`	
Explicar, sem consultar, o que significa o p95 medido	—	

7. O QUE FOI CONSTRUÍDO ATÉ ESTA DATA / DIFICULDADES ENCONTRADAS

Descrever o que já funciona, o que falta e os problemas enfrentados.

8. VISTO DO PROFESSOR

Situação da etapa	( ) Adequada ( ) Adequada com ressalvas ( ) Insuficiente ( ) Não apresentada

Observações:

9. ASSINATURAS

Aluno(a) 1	Aluno(a) 2	Aluno(a) 3
Aluno(a) 4	Aluno(a) 5	Professor(a) supervisor(a)

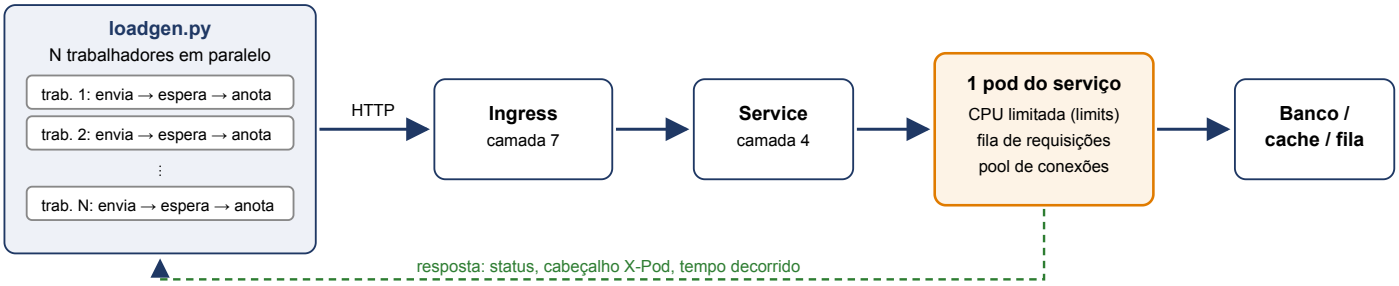
ANEXO A – O QUE CONSTRUIR NA ETAPA 4 E O QUE O GERADOR PROVOCA NO CLUSTER

Leitura obrigatória antes de começar o código. Responde a quatro perguntas: **o que o script faz**, **o que ele força** no cluster, **o que se espera observar** e **o que o grupo entrega**.

A1. PARA QUE SERVE O GERADOR DE CARGA

O documento da APS pede que o grupo demonstre "**com dados medidos e não com afirmações**" que o sistema escala, distribui carga, tolera falhas e é atualizado sem interrupção. Nenhuma dessas propriedades aparece com um usuário clicando no cliente web: um serviço com uma réplica responde em poucos milissegundos a uma pessoa. As propriedades só aparecem **sob pressão**, com centenas de requisições por segundo chegando ao mesmo tempo. O gerador de carga é essa pressão, controlada e medida: um **"público artificial"** de tamanho conhecido, que também funciona como **cronômetro**. É o instrumento de medição de toda a APS, e por isso precisa ser escrito pelo grupo, que deve saber exatamente o que ele mede.

A2. O QUE O SCRIPT FAZ, PASSO A PASSO



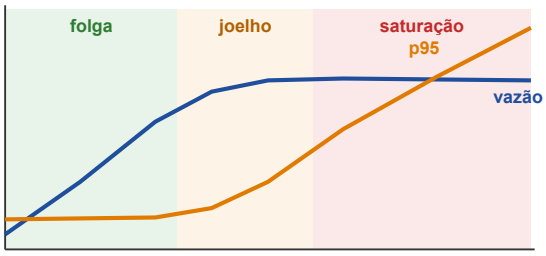
#	O SCRIPT…	DETALHE
1	Lê os parâmetros da linha de comando	URL alvo (sempre pelo **Ingress**), verbo, corpo, nº de trabalhadores, duração, modo (constante, rampa ou degraus), *timeout*, prefixo dos arquivos de saída.
2	Cria N **trabalhadores** (threads ou tarefas assíncronas)	Cada trabalhador é **um cliente simulado**, como um sensor ou usuário, num **laço fechado**: envia uma requisição, **espera a resposta** e só então envia a próxima. N trabalhadores = N requisições em andamento ao mesmo tempo.
3	Monta cada requisição	Para a ingestão: `POST` com uma leitura de sensor em JSON, **variando** estação e valor (dados repetidos podem ser otimizados ou rejeitados). Para a consulta: `GET` da rota agregada.
4	Cronometra e anota **cada** requisição	Hora de início, latência (do envio até o último byte da resposta), código HTTP, valor do cabeçalho `X-Pod` (e `X-Versao`), tipo de erro (`http_503`, `timeout`, conexão recusada). Nenhuma requisição é descartada, nem as que falharam.
5	Controla a intensidade ao longo do tempo	**Constante**: N trabalhadores do início ao fim. **Rampa**: começa com 1 e chega a N em S segundos (usado no HPA, Etapa 5). **Degraus**: 1, 2, 4, 8… trabalhadores, cada degrau por um tempo fixo (usado na linha de base).
6	Mostra o progresso a cada segundo	Uma linha por segundo com hora, trabalhadores ativos, req/s, erros, p50, p95 e nº de pods distintos que responderam.
7	Calcula o resumo ao final	Ordena as latências e extrai **p50, p95 e p99**; calcula **vazão** (respostas OK por segundo) e **taxa de erro**; conta respostas **por pod** e por código HTTP. Com degraus, um resumo por degrau, descartando o aquecimento.
8	Grava os resultados	CSV com **uma linha por requisição** (matéria-prima), CSV com **uma linha por segundo** (para as linhas do tempo da Etapa 5) e JSON com o resumo (para as tabelas do relatório).

A3. O QUE A CARGA FORÇA NO CLUSTER

O que acontece, componente por componente, quando o número de trabalhadores sobe com uma única réplica do serviço.

COMPONENTE	O QUE A CARGA PROVOCA	COMO O GRUPO ENXERGA
Pod do serviço: CPU	O consumo sobe até o `limits.cpu` do manifesto. Atingido o limite, o kernel **estrangula** o contêiner (*CPU throttling*): o pod não ganha mais processador, e cada requisição passa a esperar a sua vez.	`kubectl top pod` parado no valor do limite (ex.: `500m`) enquanto a latência sobe
Pod do serviço: fila	Requisições que chegam mais rápido do que são atendidas **acumulam** em fila. O tempo na fila soma-se à latência: é por isso que, depois da saturação, a latência cresce e a vazão não.	p50 e p95 subindo degrau a degrau com vazão constante
Banco de dados	Cada requisição de ingestão é uma escrita. O banco é **único** (StatefulSet) e tem o seu próprio limite de CPU e de conexões; pode saturar **antes** do serviço. Na Etapa 5, isso limita o ganho de adicionar réplicas.	`kubectl top pod banco-0`; erros de conexão nos logs
Fila de mensagens / cache	Se o consumidor (serviço de alertas) for mais lento que a ingestão, as mensagens **acumulam** na fila. No cache, a taxa de acerto define quanto trabalho chega ao banco.	Tamanho da fila (painel do RabbitMQ, `XLEN` / `LLEN`)
Probes	Sob carga extrema, a *readiness* pode demorar mais que o seu `timeoutSeconds` e falhar: o pod **sai do balanceamento** e o gateway devolve `503`. Se a *liveness* falhar, o pod é **reiniciado** no meio do teste.	`kubectl get pods -w` (`0/1`, `RESTARTS`); `kubectl describe pod`
Ingress (gateway)	Mantém as conexões e repassa as requisições; com *timeout* do lado dele, responde `504`. Com 2+ réplicas, distribui em rodízio: o `X-Pod` prova isso.	Contagem “por pod” do resumo; códigos 502/503/504
HPA	**Desligado nesta etapa, de propósito.** Se estiver ligado, cria réplicas no meio do teste e a medição deixa de ser de “uma réplica”. Ele volta na Etapa 5.	`kubectl get hpa` sem recursos
Máquina hospedeira	Cluster, banco, gateway **e o próprio gerador** disputam os mesmos núcleos do notebook. O gerador também pode ser o gargalo.	Monitor de atividade / `top` da máquina

A4. O QUE SE ESPERA OBSERVAR: A CURVA DE SATURAÇÃO



trabalhadores (1, 2, 4, 8, 16, 32, 64)

REGIÃO	O QUE ACONTECE
Folga	Dobrar os trabalhadores **dobra a vazão**; a latência quase não muda. O pod tem CPU sobrando.
Joelho	A vazão para de crescer; a latência começa a subir. Algum recurso chegou ao limite. **É o ponto de saturação que a APS pede.**
Saturação	Mais trabalhadores só aumentam a **fila**: a latência cresce na proporção dos trabalhadores (Lei de Little), aparecem *timeouts*, `503` e `504`.

O resultado da etapa é uma frase com números, por exemplo: “Com uma réplica (limite de 500m de CPU), o serviço de ingestão satura em ~16 trabalhadores, com vazão máxima de 220 req/s e p95 de 85 ms; acima disso, a vazão fica constante, o p95 passa de 300 ms e, com 64 trabalhadores, 2% das requisições falham por *timeout*. O recurso que satura é a CPU do pod, que fica em 500m.” *(Os números são ilustrativos.)* Na Etapa 5, cada experimento será comparado a essa frase: com 2 réplicas a vazão chegou a 440? Com o HPA, quanto tempo o p95 ficou acima de 85 ms?

A5. O QUE O GRUPO PRECISA ENTREGAR NESTA ETAPA

#	ENTREGÁVEL	CONTEÚDO MÍNIMO
1	`carga/loadgen.py`	O gerador da seção A2, autoral, com os requisitos do checklist (item 4 da ficha). Pode usar `threading`, `asyncio`, `requests`, `httpx` ou `aiohttp`; não pode ser só uma chamada ao Locust ou ao k6.
2	Escolha das rotas-alvo	No mínimo a rota de **ingestão** (`POST` de leituras, o serviço que o HPA vai escalar) e uma rota de **consulta**. Justificar a escolha em 3 a 5 linhas: qual recurso cada rota consome (CPU, banco, fila).
3	Procedimento da linha de base	1 réplica de cada serviço; HPA removido; degraus 1, 2, 4, 8, 16, 32 e 64 trabalhadores (ou até saturar); 20 a 30 s por degrau; 5 s de aquecimento descartados; `kubectl top pod` anotado em cada degrau; ao menos 3 repetições.
4	`resultados/base/`	Os arquivos brutos de cada execução (CSV por requisição, CSV por segundo, JSON de resumo) e o comando exato usado.
5	Tabela e gráfico	A tabela da seção 5 da ficha e um gráfico de latência (p50, p95, p99) e vazão por nº de trabalhadores, com a região de saturação indicada.
6	Análise (rascunho do relatório)	Onde fica o joelho; qual recurso saturou e qual a evidência; se o gerador pode ter sido o limite; a frase-régua da seção A4.

Limite ético e técnico: o gerador é apontado **somente para o cluster do próprio grupo**, em `localhost`. Disparar carga contra serviços de terceiros, inclusive APIs públicas usadas como fonte de dados, é ataque de negação de serviço e não faz parte da APS.

ANEXO B – GUIA RÁPIDO DO GERADOR DE CARGA E DA LINHA DE BASE

Material de apoio ao preenchimento da ficha da **Etapa 4**. Detalhamento completo na **Aula Prática 4 – Gerador de carga em Python e linha de base**.

A. VOCABULÁRIO DA MEDIÇÃO

TERMO	SIGNIFICADO	POR QUE IMPORTA
Trabalhador	Um cliente simulado em laço fechado: envia, espera a resposta, envia de novo	A concorrência é o número de trabalhadores, não de requisições por segundo
Vazão (*throughput*)	Respostas bem-sucedidas por segundo	Mede a capacidade do sistema
Latência	Tempo entre enviar a requisição e receber a resposta completa	Mede o que o usuário sente
p50 / p95 / p99	Latência abaixo da qual estão 50%, 95% e 99% das requisições	A média esconde os lentos; o p95 mostra o pior caso frequente
Taxa de erro	Requisições com status ≥ 400, *timeout* ou falha de conexão, sobre o total	Vazão alta com erro alto não é capacidade
Aquecimento	Segundos iniciais descartados de cada degrau	Conexões abrindo e JIT compilando distorcem o início
Saturação	Ponto em que mais carga não aumenta a vazão, só a latência (o "joelho" da curva)	É o número de referência para toda a Etapa 5

B. LEI DE LITTLE – A CONTA QUE CONFERE A MEDIÇÃO

Trabalhadores = vazão × latência média (em laço fechado). Ex.: 16 trabalhadores e latência média de 80 ms ⇒ vazão ≈ 16 / 0,080 = **200 req/s**. Se a vazão medida for muito diferente disso, há erro na medição. Consequência: depois da saturação, dobrar os trabalhadores **dobra a latência** e mantém a vazão.

C. OS ERROS QUE MAIS INVALIDAM A LINHA DE BASE

ERRO	CONSEQUÊNCIA	CORREÇÃO
HPA ligado durante a linha de base	Réplicas aumentam no meio do teste; a base deixa de ser "1 réplica"	`kubectl delete hpa` e `kubectl scale --replicas=1`
Só a média de latência	Picos invisíveis; análise rasa	Sempre p50, p95, p99 e taxa de erro
Gerador saturado	Mede-se o limite do script, não o do sistema	Observar a CPU da máquina; comparar com a Lei de Little
Uma única execução	Variação aleatória confundida com efeito	Repetir ao menos 3 vezes e informar a variação
Rota que muda com o tempo	Ex.: listar todas as leituras fica mais lento a cada teste	Paginar (`LIMIT`) ou medir rota de custo estável
Erros descartados do resumo	Latência "ótima" porque as lentas viraram *timeout*	Reportar a taxa de erro ao lado de cada latência

D. EVIDÊNCIAS A GUARDAR DESDE JÁ PARA O RELATÓRIO FINAL

- Código do gerador e o comando exato de cada execução (com todos os parâmetros).
- Arquivos brutos: CSV por requisição, série por segundo e resumo JSON de cada execução.
- Tabela e gráfico da linha de base; saída de `kubectl top pod` no ponto de saturação.
- Descrição do ambiente: CPU, núcleos, memória, cluster, nº de nós, onde rodava o gerador.